

GABRIEL POPA

Senior Software Engineer - Go • AWS

Cluj, Romania | +40 764309991 | gabrielpdev@outlook.com | <https://www.linkedin.com/in/gabriel-popa-12890b414/>

ABOUT ME

Senior Software Engineer with **10+ years of experience** building **backend platforms**, **API-driven systems**, and scalable business applications across product engineering, internal operations, and data-heavy workflows. Strong specialization in **Go**, **distributed services**, **database-intensive features**, and **cloud-oriented delivery**, with hands-on work across **Go 1.12–1.22**, **Gin**, **gRPC**, **PostgreSQL**, **Redis**, **Docker**, and **Kubernetes** in one continuous engineering path. Background includes **logistics**, **operational software**, and **enterprise workflow platforms**, with consistent ownership of **feature delivery**, **legacy cleanup**, **bug resolution**, **performance tuning**, and **service modernization**.

SKILLS

- **Languages:** Go 1.12–1.22, SQL, JavaScript, TypeScript, Bash
- **Backend & APIs:** Go, Gin, Echo, net/http, REST APIs, gRPC, microservices, background workers, WebSocket basics, authentication, authorization, RBAC, API validation, file processing, concurrency patterns, middleware design
- **Frontend:** HTML5, CSS3, JavaScript, TypeScript, React basics, server-rendered admin pages, dashboard integration
- **Data & Messaging:** PostgreSQL, MySQL, SQL Server, Redis, MongoDB basics, Elasticsearch, RabbitMQ, Kafka basics, ETL support, reporting queries, indexing, query tuning, caching strategies
- **Cloud & DevOps:** Docker, Kubernetes, Helm, Git, GitHub Actions, GitLab CI, Jenkins, Linux, Nginx, Terraform basics, AWS basics, CI/CD pipelines, release automation, containerized deployment
- **Observability / Quality / Security:** Prometheus, Grafana, OpenTelemetry basics, structured logging, error tracing, unit testing, integration testing, API testing, load-test support, JWT, OAuth basics, rate limiting, audit logging, secure configuration handling

PROFESSIONAL EXPERIENCE

Senior Software Engineer

Soft Build | Bucharest, Romania (Jan2024 – Dec 2025)

- Led delivery of backend modules for an operational platform using **Go 1.21–1.22**, **Gin**, **PostgreSQL**, and **Redis**, translating fragmented **reporting** and **administration requirements** into cleaner **APIs** and more stable **service behavior** across internal teams.
- Modernized older **service logic** that had grown difficult to extend by splitting tightly coupled handlers into smaller **domain-focused packages**, which improved **maintainability** and reduced **regression issues** during fast feature delivery by **18%**.
- Built reusable **REST endpoints** for **dashboards**, **approval flows**, **exports**, and **search-heavy screens**, giving frontend teams a more predictable contract and reducing repeated **business-rule duplication** across multiple modules.
- Solved a recurring **production timeout** affecting larger dashboard requests by profiling slow **SQL paths**, adding missing **indexes**, and introducing safer **pagination patterns**, which improved **response time consistency** by **31%**.
- Implemented a new **permission-aware administration flow** with **JWT-secured APIs** and **role-based checks**, preventing accidental overreach in sensitive modules and reducing manual review work for internal operators.
- Stabilized a difficult **export issue** where large **CSV jobs** failed under memory pressure by moving generation into **streaming-based handlers** and **background execution**, which improved **completion reliability** by **27%**.
- Introduced **structured logging** and **trace-friendly request identifiers** around critical endpoints, making it easier to connect user-reported failures with actual backend behavior during **production debugging**.
- Supported migration of selected services from older deployment routines into **Docker-based releases** with cleaner **environment separation**, reducing rollout inconsistency and making non-production verification more dependable.

- Improved **cache behavior** for hot endpoints by adding targeted **Redis invalidation rules** instead of coarse cache clears, which reduced **stale-data complaints** without losing the performance benefit of cached reads.
- Worked closely with **QA** and **product teams** to reproduce edge-case defects around **approval sequencing**, then resolved them through **transaction ordering fixes** and stronger **validation rules** in write-heavy API flows.
- Added **health checks**, **startup validation**, and safer **config handling** for service initialization, preventing several avoidable release-time failures caused by missing variables and partial environment setup.
- Mentored engineers on practical **Go patterns** such as **interface boundaries**, **error wrapping**, and **concurrent worker design**, helping the team keep new code more consistent during a fast-moving release cycle.

Software Engineer

Next Apps | Ghent, Belgium (*Nov2020 – Nov 2023*)

- Built and maintained backend services for workflow-oriented business applications using **Go 1.17–1.20**, **Echo**, **Gin**, **PostgreSQL**, and **Docker**, supporting internal tools that required reliable **APIs**, **imports**, **approval rules**, and **reporting features**.
- Delivered new **API-driven modules** for **data intake**, **task assignment**, and **document-linked processing**, helping replace several manual steps with stable backend workflows that improved **operational throughput** by **22%**.
- Refactored older **monolithic handlers** into clearer **service** and **repository layers**, which made **business rules** easier to test and reduced the chance of breaking unrelated endpoints during enhancements.
- Solved an intermittent bug where users saw **duplicate records** after retries by tightening **idempotency checks** and improving **request validation**, which reduced data correction tickets raised by support teams.
- Implemented **asynchronous processing** for heavier **import jobs** using **worker-based patterns** and queued execution, preventing long-running requests from blocking normal user operations during peak usage periods.
- Improved **database performance** on reporting-heavy screens by rewriting **joins**, reducing unnecessary payload size, and adding better **index coverage**, which lowered average **query latency** by **26%**.
- Helped migrate selected integrations from older **JSON parsing** and manual mapping code into cleaner **typed models**, reducing parsing-related **runtime errors** and making downstream logic easier to maintain.
- Introduced **containerized local development** and **test environments** with **Docker Compose**, making onboarding smoother and reducing the “**works on my machine**” issues that had slowed team delivery.
- Added more structured **error responses** and **log context** to API failures, which allowed frontend and QA teams to isolate **validation issues** faster instead of escalating generic backend incidents.
- Supported rollout of internal **audit logging** on sensitive update paths, improving **change traceability** for admin actions and making production issue review more straightforward for engineering and operations.
- Worked flexibly across **backend delivery**, **SQL troubleshooting**, and small **frontend integration tasks** when releases became blocked, helping keep deadlines realistic without forcing risky last-minute code changes.

Software Developer

Datamart | Stockholm, Sweden (*Feb 2016 – Sept 2020*)

- Contributed to backend and data-centric application development using **Go 1.13–1.16**, **SQL**, **server-side integrations**, and **reporting workflows**, with a strong focus on **reliability**, **maintenance**, and incremental platform improvement.
- Built **service endpoints** and **scheduled processing jobs** that supported operational data flows, reducing repetitive manual handling and giving internal users more dependable access to current business information.
- Improved a fragile **import pipeline** that frequently failed on malformed input by adding safer **parsing**, **row-level validation**, and clearer **rejection logging**, which reduced support escalations by **19%**.
- Helped replace repeated **raw query logic** with better-structured **repository patterns** and shared helpers, making future **bug fixes** easier and reducing inconsistencies across modules touching the same tables.
- Investigated a recurring issue where **reporting totals** drifted from source records under concurrent updates, then resolved it by tightening **transaction usage** and correcting **aggregation logic** in affected paths.
- Supported gradual adoption of **Go** for backend utility services alongside existing systems, using it where **performance**, **simplicity**, and **deployment ease** made sense without forcing unrealistic full rewrites.
- Improved **release confidence** by adding more repeatable **smoke-test steps** and **integration checks** for critical endpoints, which reduced post-release defect discovery in common operational flows.
- Worked on **search** and **filtering improvements** for admin tools by refining **SQL predicates** and **response shaping**, helping users navigate larger datasets without overwhelming pages with unnecessary payloads.

- Assisted with **service deployment scripting**, **Linux configuration adjustments**, and basic **monitoring checks**, building stronger operational awareness while supporting senior engineers on production rollouts.
- Collaborated with **analysts** and **support staff** to reproduce **business-rule defects** from real cases, then translated those findings into safer backend fixes rather than surface-level UI-only patches.

Junior Software Developer

Berg Software | Timișoara, Romania (Sep2014 – Mar 2016)

- Supported backend and web application development using early **Go 1.4–1.6**, **SQL**, **JavaScript**, **HTML**, and **CSS**, contributing mainly to **bug fixes**, **small features**, **admin tools**, and **maintenance work** under senior guidance.
- Built smaller **service endpoints** and **helper utilities** for internal modules, gaining hands-on experience with **request handling**, **validation**, and clean separation between **transport logic** and **business rules**.
- Fixed a recurring **form-processing problem** caused by mismatched validation between frontend input rules and backend checks, which reduced failed submissions and improved **QA pass rates** by **14%**.
- Helped troubleshoot an intermittent **login issue** related to **session handling** and **environment-specific configuration**, and the final fix made **authentication behavior** more predictable across staging and production.
- Worked on basic **data migration** and **correction scripts** when legacy records caused downstream application errors, supporting smoother releases by removing problematic edge cases before deployment windows.
- Improved maintainability in older modules by extracting shared **utility functions** and reducing duplicated logic, which made later enhancements simpler for more senior developers to implement safely.
- Assisted with **SQL query adjustments** and small **indexing improvements** for internal dashboards, helping pages remain usable as record counts started to grow beyond the original assumptions.
- Participated in debugging **production issues** by reproducing defects locally, reviewing logs carefully, and delivering focused patches instead of risky broad refactors that were not suitable for an early-career role.
- Contributed to **documentation** for setup, release steps, and common issue handling, which improved team handoff and made repeated support tasks easier to execute consistently.

EDUCATION

Bachelor's Degree in Computer Science

University of Bucharest | (2010 – 2014)

- Academic foundation covered software engineering, algorithms, database systems, and practical application development, providing the base for long-term specialization in backend engineering and scalable service design.

CERTIFICATIONS

Go: Building Microservices — 2024

- Demonstrated practical capability in designing **service boundaries**, handling **inter-service communication**, and building maintainable **backend systems** with **Go**.

Docker & Kubernetes Foundations — 2023

- Validated working knowledge of **containerized deployment**, **runtime isolation**, and basic **orchestration patterns** commonly used for modern backend service delivery.

PostgreSQL for Developers — 2022

- Demonstrated hands-on understanding of **relational modeling**, **query optimization**, **indexing**, and **performance-aware backend data access patterns**.